

Dragon Nine — Content Scraper Integration Guide

studio_project (/home/tim/studio_project)

1. FILE PLACEMENT

Copy these files into your project:

```
studio_project/  
├─ src/  
│   ├── components/  
│   │   └─ ContentScraperMonitor.jsx      - NEW (dashboard module)  
│   ├── services/  
│   │   └─ d9-scheduler.js                - NEW (if using server-side D9 scheduling)  
│   └─ ...  
├─ server/  
│   └─ routes/  
│       └─ scraper.js                     - NEW (Express API routes)  
├─ scraper/  
│   └─ youtube-ingest.js                  - NEW (from previous output)  
├─ data/  
│   └─ scraped/                           - NEW (mkdir -p, gitignore this)  
└─ vite.config.js                         - MODIFIED (proxy /api to server)
```

2. VITE PROXY CONFIG

Add to vite.config.js:

```
export default defineConfig({  
  // ... existing config ...  
  server: {  
    port: 5173,  
    proxy: {  
      '/api': {  
        target: 'http://localhost:3001', // your Express server port  
        changeOrigin: true,  
      },  
    },  
  },  
});
```

3. EXPRESS SERVER WIRING

In your Express server entry (e.g., server/index.js or api/server.js):

```
const scraperRoutes = require('./routes/scraper');
```

```
// Mount scraper API
app.use('/api/scraper', scraperRoutes);
```

Make sure YOUTUBE_API_KEY is in your .env:

```
YOUTUBE_API_KEY=your_api_key_here
```

4. DASHBOARD INTEGRATION

Import and mount the monitor component in your Sovereign Dashboard layout:

```
import ContentScraperMonitor from './components/ContentScraperMonitor';

// In your dashboard grid/panel:
<div className="dashboard-panel scraper-panel">
  <ContentScraperMonitor />
</div>
```

Recommended panel sizing: col-span-2 row-span-2 (or full width on mobile).

5. D9 TEMPORAL SCHEDULE

The scheduler auto-registers these jobs on server start:

Category	Standard Cron	D9 Equivalent	Real Interval
Underground Music	0 */3 * * *	Every ~3.33 D9h	3 hours
NZ Surf & Bodyboard	0 */4 * * *	Every ~4.44 D9h	4 hours
Extreme Outdoors	0 */6 * * *	Every ~6.67 D9h	6 hours

All times are NZST (Pacific/Auckland). The scheduler logs in D9 temporal time and respects Apex periods (9° peak at D9 hours 9 and 22).

6. TELEMETRY HEARTBEAT ALIGNMENT

The monitor component polls every 8 seconds to match your existing telemetry panel jitter. The D9 scheduler also logs status every 8 seconds.

To align scraper runs with Apex periods (for “divine connection” scheduling):

```
// In d9-scheduler.js, modify the schedule method:
if (d9Time.apex) {
  console.log(`$${vcenter{\\hbox{\\includegraphics[height=\\ht\\strutbox]{\\usr/share/docminer/emoji/
↪ 1f525.png}}}}}$ Apex period detected — prioritizing sacred`);
  // Boost scores for spiritual/meditation content
}
```

7. APPROVAL WORKFLOW

The queue endpoint (GET /api/scrapper/queue) returns items with scores 0.60–0.85. The monitor renders approve/skip buttons that call:

```
POST /api/scrapper/approve
{ videoId: "...", action: "approve" | "reject" }
```

Items 0.85 are auto-published to the broadcast stage. Items <0.60 are archived.

8. SATELLITE SYNDICATION

The broadcast manifest includes a bySatellite map. Wire this into your mesh gateway to push content automatically:

```
// In your mesh gateway service
const manifest = require('./data/scraped/latest-manifest.json');

manifest.bySatellite.club.forEach(videoId => {
  pushToSatellite('club', videoId);
});
```

9. DEPENDENCIES

```
npm install cron fast-xml-parser node-fetch lucide-react
```

lucide-react is likely already installed. cron is new for the scheduler.

10. ENVIRONMENT CHECKLIST

- YOUTUBE_API_KEY set in .env
 - data/scraped/ directory exists (mkdir -p)
 - data/scraped/ in .gitignore
 - Express server running on port 3001 (or update Vite proxy)
 - Vite dev server proxy configured for /api
 - ContentScraperMonitor imported in dashboard
 - D9Scheduler imported and started in server entry
 - Scraper API routes mounted at /api/scrapper
-

11. D9 CLOCK DISPLAY

The monitor shows live D9 time in the header. To sync with your existing D9 Clock component, expose a shared temporal utility:

```
// src/lib/d9-time.js
export function getD9Time(date = new Date()) {
```

```
const D9_HOUR = 54;
const totalMin = date.getHours() * 60 + date.getMinutes();
const d9Min = (totalMin / 1440) * (26.667 * D9_HOUR);
return {
  hour: Math.floor(d9Min / D9_HOUR) % 27,
  minute: Math.floor(d9Min % D9_HOUR),
  second: date.getSeconds(),
};
}
```

Share this between D9Clock.jsx and ContentScraperMonitor.jsx.

12. NEXT STEPS

1. Drop the files in place
 2. `npm install cron`
 3. Add the Vite proxy
 4. Mount the routes
 5. Import the component
 6. Restart dev server
 7. Hit the dashboard — you should see the scraper monitor live
 8. Click “Run All Scrapers” to test the full pipeline
-

Built for Dragon Nine · Fluid Motion of the Future